Opensource your Quant Strategy

As of Oct 24, 2026

aiInvesting: an AI investment team you can run on your own laptop

No API keys. Free data. A free open-weight model. Everything open source.

Seven agents do the judgment. Plain Python does the schedule, the arithmetic and the bookkeeping.
A coach reviews every quarter and the dashboard shows whether the team is getting better.

Code, spec and slides: https://github.com/jimkliew/aiInvesting MIT license

The question we are asking

A lab, not a product

How well can an AI investment team trade, and does it get better with feedback?

1 Can it do quant research?

Agents propose hypotheses, write strategy code, backtest it, and critique the result. Measured against buy-and-hold and a moving-average crossover.

2 Can it trade with judgment?

A trader persona makes weekly BUY, SELL or HOLD calls from a point-in-time snapshot and must state its reason in one sentence.

3 Does coaching help?

After each quarter a Review Agent studies the results and recommends changes. The next quarter shows whether they helped.

Success is measured one way: the team's out-of-sample track record against SPY, season by season.

SPY is the benchmark the team must beat. It never holds SPY.

The team trades twelve large-cap US names: AAPL, MSFT, NVDA, AMZN, GOOGL, META, TSLA, AVGO, AMD, CRM, ORCL, NFLX.

Assumptions: free, local, and keyless

Everything a student needs is a laptop and one download

$0

to run the whole lab, end to end

0

API keys, accounts or subscriptions

13

tickers: 12 traded, plus SPY as the benchmark

16 GB

of laptop RAM runs the default model

WHAT WE RELY ON

1 Data: yfinance, daily bars

Free, no key. Daily open, high, low, close and volume from 2015. Adjusted for splits and dividends. The Data Agent checks gaps and short histories and writes a report.

2 Model: Ollama with qwen3:8b

Open weights under Apache 2.0, runs fully offline. Supports tool calling and JSON-schema output. Swap any Ollama model with one flag; qwen3:4b fits 8 GB laptops.

3 Compute: your own machine

About 2 to 5 seconds per model call on Apple Silicon. A six-season campaign is roughly 300 calls, 20 to 30 minutes. Replay mode shows a recorded run in class.

4 Market model: deliberately simple

US large caps, daily closes, long only, monthly desk rebalance (weekly is a coach lever), 5 basis points per unit of weight traded. No shorting, leverage, options or intraday data.

yfinance is an unofficial interface to Yahoo Finance; use it for personal and educational purposes and expect occasional gaps. Not investment advice.

The team: seven roles and one orchestrator

Each agent has one prompt file, one output schema, and at most four tools

> The model does the judgment. Plain Python does the arithmetic, the schedule and the bookkeeping.

1 Data Agent

Turns a one-line brief into a checked, cached dataset with a manifest hash and a short report.

2 Idea Agent

Reads the lab notebook and proposes distinct strategy hypotheses across families.

3 Coder Agent

Writes generate_signals() for one hypothesis and fixes it when the sandbox objects.

4 Critic Agent

Reads train and validation metrics and votes Keep or Discard, naming its concerns.

5 Trader personas

Momentum, contrarian, risk-averse. Weekly calls from a snapshot with one sentence of reasoning.

6 PM Agent, optional

Combines the three personas' votes into one allocation each week. The committee pattern.

7 Review Agent, the coach

Studies each season and recommends changes, but only from a fixed menu of levers.

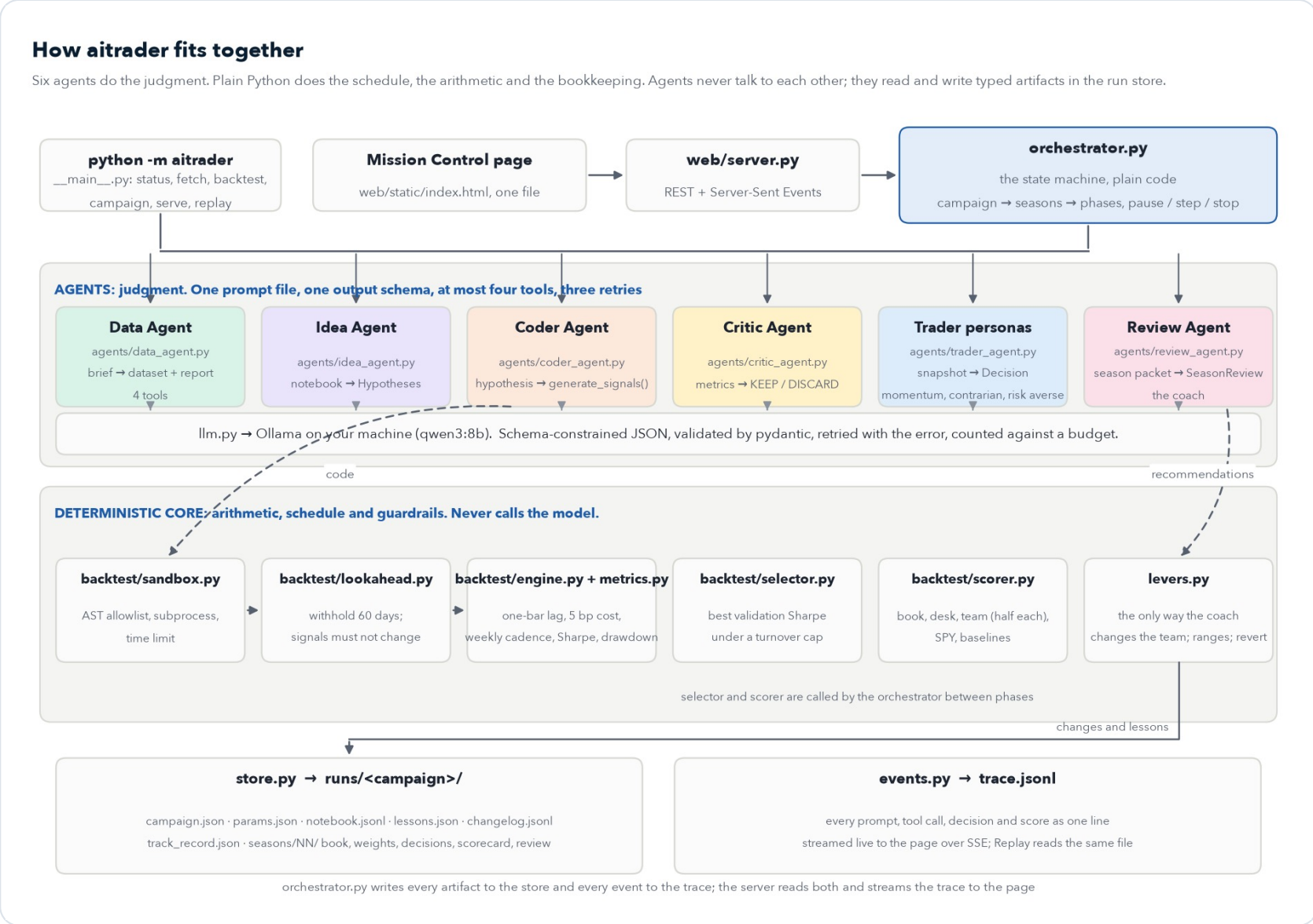
py Orchestrator, plain code

A state machine, not a model. Runs the phases in order and enforces every budget.

Only the Data Agent uses tool calling. The other six get everything they need in the prompt, which is cheaper and more reliable for an 8B model.

How it all fits together

Agents do the judgment; plain Python does the schedule, the arithmetic and the bookkeeping



docs/architecture.png in the repo, drawn by docs/draw_architecture.py. docs/ARCHITECTURE.md adds the agent table, the on-disk layout and where to change things.

How a season runs

One calendar quarter the team has never seen, traded once

Research on the past, trade the sealed quarter, score it, get coached, repeat.

1 Research

Idea, Coder and Critic loop over hypotheses using only data from before the quarter.

2 Select

Code, not a model, picks the best validation Sharpe under a turnover cap. That is the quant book.

3 Trade

The quant book trades mechanically. The desk persona makes a discretionary call per stock at each rebalance.

4 Score

Quant book, desk, team (half each), SPY and baselines. The track record grows by one quarter.

5 Review

The coach reads the season packet and writes an assessment with 1 to 3 recommendations.

6 Apply

Levers are applied automatically, or wait for you to click Accept. The next season begins.

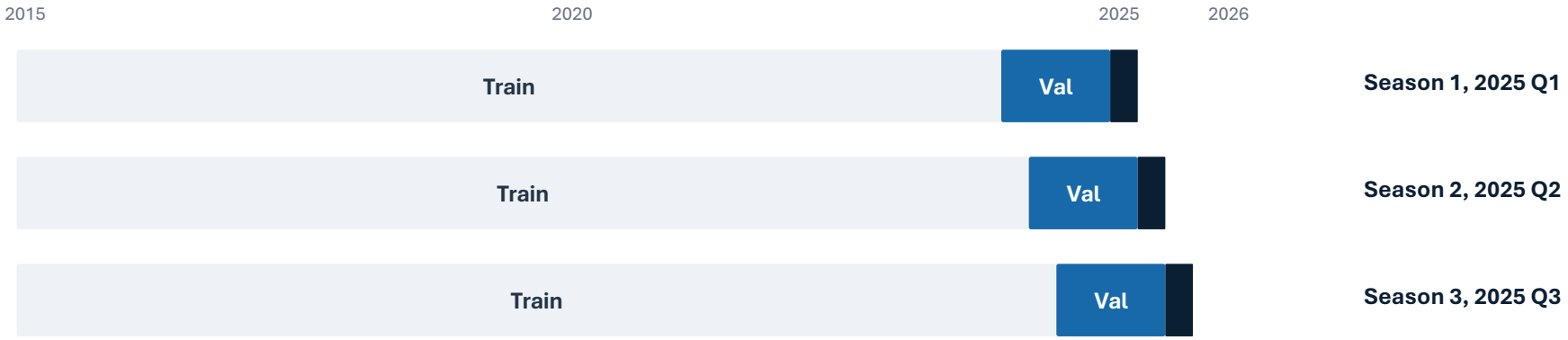
Default campaign: 6 seasons from 2025 Q1 to 2026 Q2, about 300 model calls, 20 to 30 minutes on a laptop.

Season 1 also runs the Data Agent and a second research round. Later seasons reuse the cached data and carry the notebook forward.
Budgets, not patience, end every loop: calls per season, minutes per season, strategies per round.

Why seasons: walk-forward testing stops the loop from peeking

A fixed test set would be burned the first time the coach saw it

Each quarter is traded exactly once, out of sample. Only then does it become history the team may learn from.



Navy block: the sealed quarter. It moves one step right each season.

TRAIN

2015 up to twelve months before the season. Every backtest runs here. The Idea, Coder and Critic agents see only this and the validation year.

VALIDATION

The twelve months before the season. The Critic judges here, and the selector picks the quant book here. Nothing from the season influences the choice.

SEASON

The next quarter. Sealed until traded, scored once, then handed to the coach as history. The next season starts a quarter later.

What to watch: did validation Sharpe predict season Sharpe? With six seasons noise dominates. The lab teaches the measurement, not a guarantee.

Data Agent

Phase 0. Runs once per campaign; later seasons reuse the cache

> Lesson one: do not use a model where a function will do. The fetch is code; the judgment around it is the agent.

ROLE

Turn a plain-English brief into a validated, cached dataset and a short written report.

INPUT

A brief such as "12 large-cap US tech names plus SPY, 2015 to today", plus a bundled list of candidate tickers. No web search.

OUTPUT

data/raw/*.parquet, data/features/*.parquet, manifest.json with a SHA-256 of the prices, and REPORT.md in plain English.

TOOLS AND LIMITS

propose_tickers, fetch_prices, check_quality, replace_ticker. At most 12 tool calls. The only agent that uses tool calling.

Prompt, excerpt

```
You assemble datasets for a long-only equity lab.
Turn the brief into a ticker list with propose_tickers, fetch with
fetch_prices, then read check_quality. Replace any name with under
8 years of history or unexplained gaps. Finish with a short report:
what was fetched, what you changed, and why. Never invent tickers;
only use names the tools return.
```

Code running

```
plan = llm.structured(DATA_PROMPT, brief, schema=DataPlan)
# -> tickers=[AAPL, MSFT, NVDA, ...], benchmark="SPY"

prices = fetch_prices(plan.tickers + [plan.benchmark],
                      start="2015-01-01")     # yfinance, parquet cache
report = check_quality(prices)                # gaps, NaNs, zero volume
manifest.write(prices, notes=plan.rationale)  # sha256 -> 3f9c...

# the tool loop underneath (aitrader/agent.py)
for step in range(max_steps):
    reply = ollama.chat(model, messages, tools=schemas(tools))
    if not reply.message.tool_calls: return reply.message.content
    for call in reply.message.tool_calls:
        messages.append(tool_result(tools[call.function.name](**call.function.arguments)))
```

Idea Agent

Phase 1. One call per research round, no tools

Everything it needs is in the prompt: a 40-line feature catalog and one line per strategy already tried.

ROLE

Propose K distinct hypotheses across families: trend, mean reversion, volatility, cross-sectional momentum.

INPUT

The feature catalog, the lab notebook summary, the family focus set by the coach, and its own lessons.

OUTPUT

A list of Hypothesis objects: name, family, idea, features_used, why_it_might_work. JSON enforced by schema.

TOOLS AND LIMITS

No tools, temperature 0. For an 8B model, putting the information in the prompt is more reliable than letting it fetch.

Prompt, excerpt

```
You are the research lead of a small quant team. Propose {K} distinct
strategy hypotheses using only the features listed. Do not repeat or
lightly vary anything in the notebook. Cover different families.
Focus: {family_focus}. Each idea must be testable as a daily, long-only
weighting rule. Answer in the JSON schema provided.
```

Code running

```
hyps = llm.structured(
    system=prompts.load("idea", lessons=store.lessons("idea")),
    user=f"Features:\n{features.catalog()}\n\nNotebook:\n{notebook.summary()}",
    schema=list[Hypothesis],
    temperature=0,
)

# under the hood, one Ollama call with schema-constrained output
ollama.chat(model="qwen3:8b", messages=[...],
            format=Hypothesis.schema(), options={"temperature": 0})

# a Hypothesis, as returned
{"name": "breakout_20d_volscale", "family": "trend",
 "idea": "Buy closes above the prior 20-day high; hold while above sma20",
 "features_used": ["close", "ret_1d"]}
```

Coder Agent

Phase 1. One function per hypothesis, at most three attempts

Every strategy, written by a person or a model, implements the same one-function interface.

ROLE

Translate one hypothesis into generate_signals() and repair it when the sandbox or the lookahead guard rejects it.

INPUT

One Hypothesis. On a retry, the exact error text and line number from the sandbox or the backtester.

OUTPUT

StrategyCode: the function body and a note. Weights between 0 and 1, one row per date, one column per ticker.

TOOLS AND LIMITS

No tools. Three attempts, then the hypothesis is marked failed in the notebook and the loop moves on.

Prompt, excerpt

```
Write the body of generate_signals(prices, features) in pandas.
Use only past data at each row. No imports, no file or network
access, no shift with a negative number. Return weights between 0
and 1 for every ticker and date. Keep it under 40 lines and comment
each indicator. If you receive an error, fix only what it names.
```

What it wrote: breakout_20d_volscale

```
def generate_signals(prices: pd.DataFrame, features: pd.DataFrame) -> pd.DataFrame:
    """Target weights in [0, 1], indexed like prices. Past data only."""
    high20 = prices.rolling(20).max().shift(1)      # yesterday's 20-day high
    sma20  = prices.rolling(20).mean()
    entry  = prices > high20                        # breakout
    exit_  = prices < sma20                         # trend lost
    held   = entry.where(entry | exit_).ffill().fillna(False)
    vol20  = features["ret_1d"].rolling(20).std() * 252 ** 0.5
    raw    = held.astype(float) / vol20             # inverse-vol sizing
    w      = raw.div(raw.sum(axis=1), axis=0).fillna(0.0)
    return w.clip(upper=0.30)                       # no name above 30%
```

Critic Agent

Phase 1. Reads numbers, not prices, and never sees the season

> Keep or Discard, with the concern named: overfitting, turnover, concentration.

ROLE

Review each backtested strategy on its train and validation metrics; flag overfitting and churn; suggest what to try next.

INPUT

The code, train metrics, validation metrics, the notebook summary, and lessons from the coach such as the turnover limit.

OUTPUT

Review: verdict KEEP or DISCARD, a list of concerns, and one suggestion. Written to the notebook with the metrics.

TOOLS AND LIMITS

No tools. The season window is still sealed when it runs, so it cannot grade on the answer.

Prompt, excerpt

```
You are a skeptical risk reviewer. Compare train and validation
Sharpe: a gap above 0.5 suggests overfitting. Discard validation
turnover above {max_turnover} percent per year. Flag any single-name
weight above 30 percent. Say what evidence would change your mind.
Lessons from the coach: {lessons}
```

Code running

```
review = llm.structured(
    system=prompts.load("critic", lessons=store.lessons("critic")),
    user=f"Code:\n{code}\n\nTrain: {m_train}\nValidation: {m_val}\n\n"
         f"Notebook:\n{notebook.summary()}",
    schema=Review,              # verdict, concerns[], suggestion
)
notebook.add(hyp, code, m_train, m_val, review)

# a Review, as returned
{"verdict": "KEEP",
 "concerns": ["top position averages 28% of the book"],
 "suggestion": "test a 25% cap; turnover 140%/yr is inside the limit"}
```

Trader personas, and the optional PM

Phase 2. One call per ticker at each rebalance, from a point-in-time snapshot

The persona sees only what existed on the decision date, about 300 tokens, and answers in a fixed JSON shape.

ROLE

Make a discretionary BUY, SELL or HOLD call with a target weight, a confidence, and one sentence grounded in the snapshot.

INPUT

Snapshot: last 20 closes, price against the 20, 50 and 200-day averages, RSI 14, returns over 1, 3 and 12 months, realized volatility, current weight.

OUTPUT

Decision: action, target_weight, confidence, reason. The engine fills at the next open and charges the cost.

TOOLS AND LIMITS

No tools. Three personas share the code and differ only by prompt file. With --committee a PM Agent merges their votes into one allocation.

Prompt, excerpt

```
You are a momentum trader managing one sleeve of a long-only
portfolio. ... Decide one action: BUY, SELL or HOLD, with a target
weight between 0 and the maximum allowed, a confidence from 0 to 1,
and one sentence grounded in the numbers shown. Never reference
information not in the snapshot.
## Lessons from the coach
1. (after S1) Do not call a stock oversold unless RSI 14 is below 30
   and it sits below its 50-day average.
```

One decision, in full

```
# what the model saw (features.snapshot("AAPL", as_of="2025-10-20"))
ticker: AAPL   as_of: 2025-10-20
closes_20d: 227.1 229.4 ... 236.8
vs_sma20: +1.9%  vs_sma50: +4.2%  vs_sma200: +9.8%   rsi14: 58
ret_1m: +3.1%  ret_3m: +11.0%  ret_12m: +19.4%   vol_20d_ann: 21%
current_weight: 0.20   max_weight: 0.30

# what it returned (schema-constrained, temperature 0)
{"action": "BUY", "target_weight": 0.25, "confidence": 0.66,
 "reason": "Broke above the 20-day high on rising volume; 3-month +11%; RSI 58."}

engine.queue("AAPL", 0.25, fill="next_open")   # filled 21 Oct, cost 0.6 bp
```

Review Agent, the coach

Phase 4. One call per season. The only agent allowed to change the team

It may recommend, but only from a fixed menu of levers, and every change is logged with a condition for reverting it.

ROLE

Assess the season in plain words, say what worked and what failed, and recommend one to three changes grounded in numbers.

INPUT

The season scorecard, the track record, the changelog, the notebook summary, and the desk's three best and three worst decisions. About 1,500 tokens.

OUTPUT

SeasonReview: assessment, what_worked, what_failed, recommendations, each with lever, target, change, rationale, expected_effect, revert_if.

TOOLS AND LIMITS

No tools. Levers: add_lesson, remove_lesson, set_param within a range, set_family_focus, retire_strategy, reinstate_strategy, revert. Universe, costs, data and model are frozen.

Prompt, excerpt

```
You coach an AI investment team. Read the season packet. Explain the
result in plain words, grounded in the numbers. Recommend at most
three changes, each using one lever from the menu, with the metric
you expect to move and the condition for reverting it. Prefer
removing a bad lesson over adding a new one. Never touch the
universe, costs, data or model.
```

Apply, then the lesson appears in the prompt file

```
review = llm.structured(system=prompts.load("review"),
                        user=packet.as_text(), schema=SeasonReview)
for rec in review.recommendations:
    change = LEVERS[rec.lever].apply(rec)   # raises if outside menu or range
    changelog.append(change, season=s)

# one recommendation, as returned
{"lever": "add_lesson", "target": "trader_momentum",
 "change": "After a drawdown over 8% reverses above the 20-day for two
            weeks, re-enter rather than wait for the 50-day.",
 "rationale": "Waiting for the 50-day cost 7.1 points on SPY in April.",
 "expected_effect": "earlier re-entry", "revert_if": "two losing re-entries"}
```

The deterministic core never calls the model

Engine, sandbox, lookahead guard, selector, scorer: plain Python you can unit-test

> If it can be a function, it is a function. The model only fills the gaps that need judgment.

Engine: tonight's signal trades tomorrow

```
def run(weights, prices, cost_bps=5):
    w = weights.clip(0, 1).shift(1)   # lag 1 bar
    w = w.fillna(0)
    ret = prices.pct_change().fillna(0)
    turn = w.diff().abs().sum(axis=1)
    cost = cost_bps / 1e4 * turn
    pnl = (w * ret).sum(axis=1) - cost
    equity = (1 + pnl).cumprod()
    return Result(equity, pnl, turn.sum())

# per window: CAGR, vol, Sharpe,
# Sortino, max drawdown, turnover, hit rate
```

Sandbox: model code never runs as-is

```
tree = ast.parse(code)
for node in ast.walk(tree):
    if is_import(node):
        raise Rejected("no imports")
    if is_call(node, "shift") and neg_arg(node):
        raise Rejected(
          f"shift(-n) reads the future, "
          f"line {node.lineno}")
    if is_call(node, ("open", "eval", "exec")):
        raise Rejected("no files, no eval")
fn = run_in_subprocess(code, timeout=5)
# allowed: pandas, numpy, math
```

Lookahead guard, selector, scorer

```
full = fn(prices, features)
cut  = fn(prices.iloc[:-60],
          features.iloc[:-60])
assert full.iloc[:-60].equals(cut)

# selector: validation only
book = max((s for s in notebook.kept()
            if s.val.sharpe >= 0.5
            and s.val.turnover <= 300),
           key=lambda s: s.val.sharpe)

# scorer: half book, half desk
team = 0.5 * book.pnl + 0.5 * desk.pnl
```

WHAT THE TESTS PROVE

Engine

A five-bar example worked by hand gives the exact equity curve, including the one-bar lag and the cost.

Sandbox

Must reject import os, open(), and shift(-1), and must time out an infinite loop at five seconds.

Lookahead guard

Must catch a strategy that deliberately peeks, and must pass every baseline.

A strategy the Coder wrote, shown on one ticker

Price, indicators, entries, exits and position size, straight from the dashboard's Strategy tab



READ THE CHART

Black is the close; blue and orange are the 20 and 50-day averages; grey steps are the previous 20-day high. Green triangles mark where the book entered, red where it exited. Shaded bands are days it held the name.

BELOW THE PRICE

The weight the book assigned to this name, then the sleeve's growth against simply holding the stock at the book's maximum weight.

WHY IT MATTERS IN CLASS

Whatever the code says, students can check it against prices day by day, switch tickers, and open the trade list to see each entry and exit.

What the tab shows: the season's quant book on one ticker. In this run the equal-weight baseline won validation every season, so the book held each of the 12 names at 8%, rebalanced weekly. Cost: 5 bp per unit of weight traded. Fill: next open, never the same close.

Strategy tab of the live page. The chart is drawn from the book's actual weights and the cached prices.

Eight rules that keep a swarm in hand

Enforced in code, not by convention

A swarm here means separation of concerns, not a free-for-all. On one laptop it is not even faster.

1 Fixed roster

Seven roles, declared in code. No agent can spawn another agent.

2 The orchestrator is code

A Python state machine runs the phases in order. No model decides who speaks next.

3 No agent-to-agent chat

Agents read and write typed pydantic artifacts on a shared run store. No conversation loops.

4 One prompt, one schema

Per agent: one prompt file, one output schema, at most four tools, at most three retries.

5 Budgets everywhere

Calls, minutes and strategies per season and per campaign. Hitting a budget stops cleanly with partial results.

6 Sequential by default

One laptop serves one model. A --parallel flag exists to prove that it buys nothing.

7 Everything replayable

Every event is appended to a JSONL trace. The dashboard replays any run at 1x, 5x or instant.

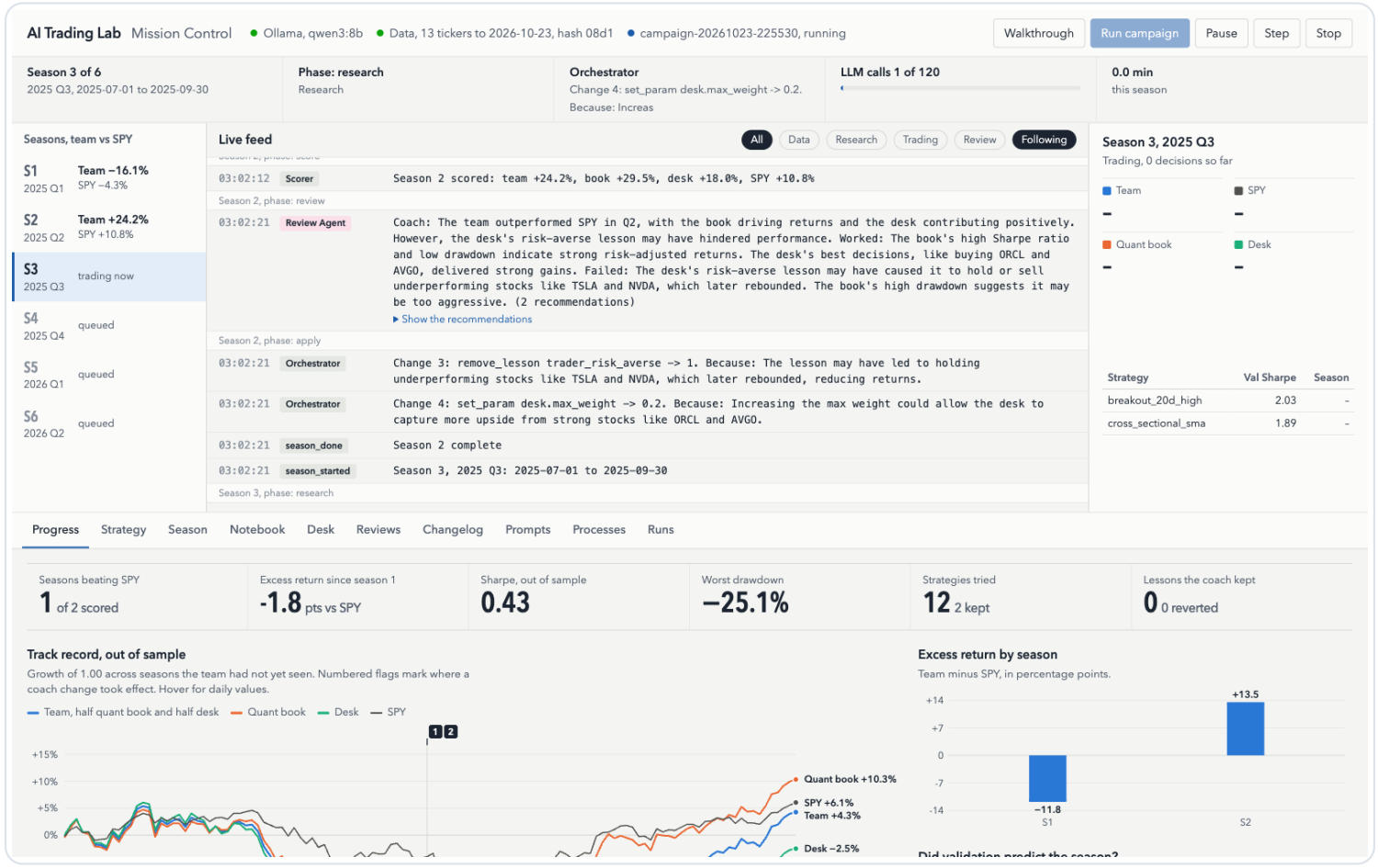
8 The coach has a menu

The Review Agent can only pull fixed levers inside allowed ranges. Universe, costs, data and model stay frozen.

Budget math on a laptop: about 50 model calls and 4 minutes per season, so 6 seasons take 20 to 30 minutes. Three personas triple the desk calls, which is why Replay exists.

Mission Control: the dashboard students run in the browser

One local page, no login. Press Walkthrough for the 17-step guided tour



WALKTHROUGH

17 steps. Each Next reveals the agent events for that stage, highlights the panel, and shows the Python running at that moment plus the background processes.

LIVE FEED AND STEP MODE

Every agent turn, tool call, decision and reason, colour-coded by agent. Pause and step one turn at a time while you narrate.

TABS FOR EVERYTHING

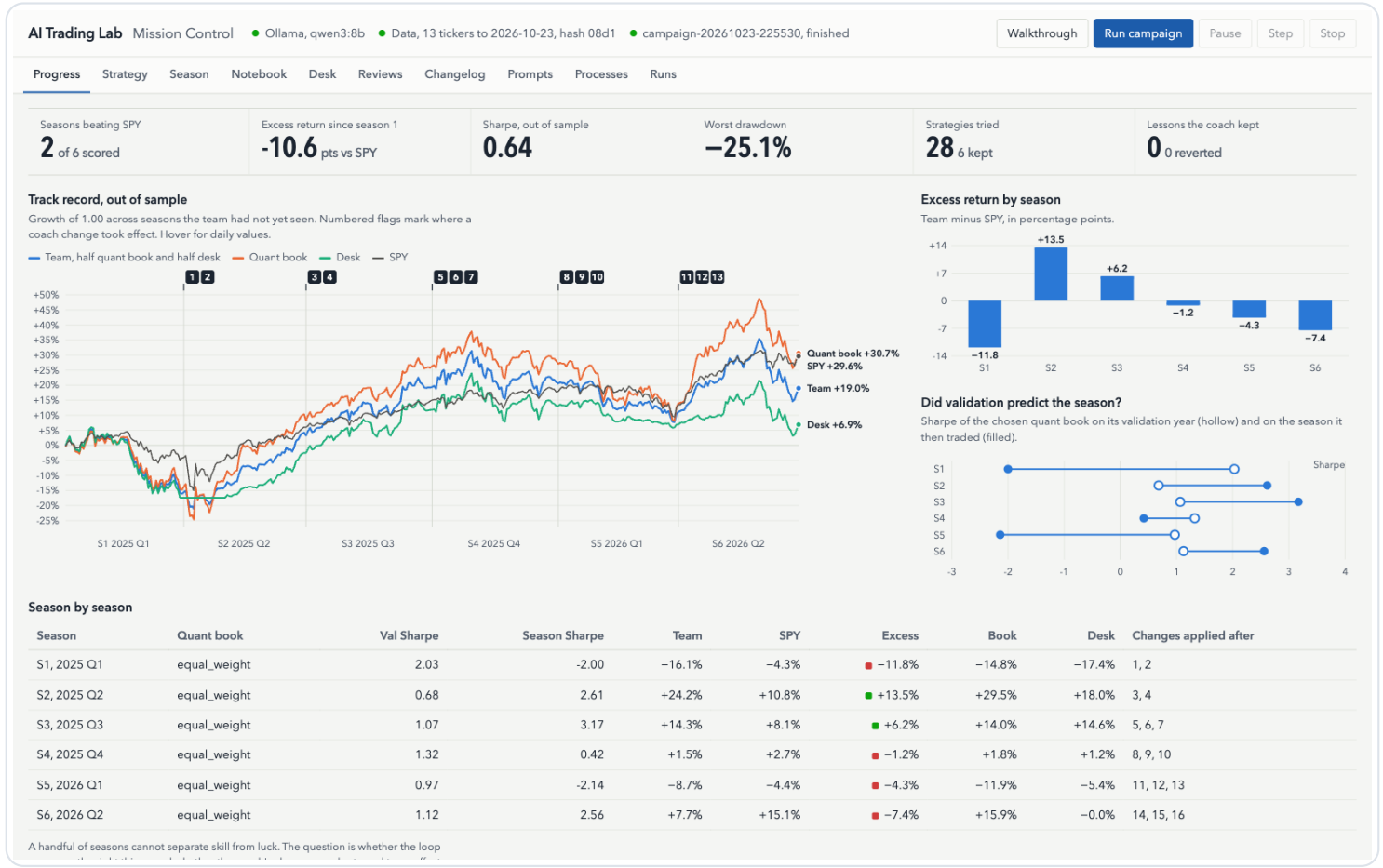
Progress, Strategy, Season, Notebook, Desk, Reviews, Changelog, Prompts, Processes, Runs. Deep links such as #reviews jump straight there.

Served by FastAPI with Server-Sent Events; one static HTML file, no build step, works offline. Start with: uv run python -m aitrader serve

Live page during a real campaign on Oct 23, 2026, qwen3:8b on a Mac Studio.

Is the team improving? The Progress tab

Track record, excess return by season, and whether validation predicted the season



TRACK RECORD

Growth of 1.00 across seasons the team had not seen, for team, quant book, desk and SPY. Numbered flags mark where each coach change took effect.

DID VALIDATION PREDICT THE SEASON?

Hollow dot: Sharpe on the validation year. Filled dot: Sharpe on the quarter it then traded. The gap is the overfitting lesson.

THE CLEANEST TEST

Run the same seasons with the coach switched off and overlay the two campaigns. The Runs tab does this with one click.

Scorecard: seasons beating SPY, excess return since season 1, out-of-sample Sharpe, worst drawdown, strategies tried, lessons the coach kept.

Real run, Oct 23, 2026. A handful of seasons cannot separate skill from luck. The lab teaches the measurement and the honest question, not a result.

Agent engineering practices you can point to in the code

Each one is named in a comment and in the README

The repo is as much about how to build agents as about trading.

1 Tools compute, the model reasons

The model never does arithmetic or touches a DataFrame.

2 Small, typed tools

At most four per agent, documented, returning compact text.

3 Schema-enforced outputs

JSON shape fixed by the model's schema mode, validated by pydantic, retried with the error.

4 Bounded loops

Max steps, wall-clock budget, tool-call cap, call budget per season.

5 Observability

Every prompt, call and decision in a JSONL trace; replayable in the dashboard.

6 No lookahead by construction

Data sliced as-of the cutoff, one-bar execution lag, perturbation test.

7 Sandboxed code

AST allowlist, no imports or files, subprocess with a time limit.

8 Evaluation first

Train, validation, sealed season. Strategies tried are counted and shown.

9 Reproducibility

Temperature 0, pinned model tag, dataset hash, prompts stored with results.

10 Memory

A lab notebook and per-agent lessons persist across seasons.

11 Human in the loop

Step mode, approve mode for coach changes, editable prompts.

12 Workflows over free-form agents

The orchestrator is code; agents do judgment inside fixed phases.

What actually happened: the first full run

6 quarterly seasons, qwen3:8b, one momentum persona, coach on, Oct 23, 2026

Out of sample, season by season. Read it as a measurement of the loop, not as a verdict on AI trading.

2 of 6
seasons the team beat SPY

+19.0%
team, compounded over the run

+29.6%
SPY over the same quarters

+30.7%
quant book alone

Season	Quant book	Val Sharpe	Season Sharpe	Team	SPY	Excess	Desk	Changes after
S1, 2025 Q1	equal_weight	2.03	-2.00	−16.1%	−4.3%	−11.8%	−17.4%	1, 2
S2, 2025 Q2	equal_weight	0.68	2.61	+24.2%	+10.8%	+13.5%	+18.0%	3, 4
S3, 2025 Q3	equal_weight	1.07	3.17	+14.3%	+8.1%	+6.2%	+14.6%	5, 6, 7
S4, 2025 Q4	equal_weight	1.32	0.42	+1.5%	+2.7%	−1.2%	+1.2%	8, 9, 10
S5, 2026 Q1	equal_weight	0.97	-2.14	−8.7%	−4.4%	−4.3%	−5.4%	11, 12, 13
S6, 2026 Q2	equal_weight	1.12	2.56	+7.7%	+15.1%	−7.4%	−0.0%	14, 15, 16

Questions for the room: did validation Sharpe predict the season? Did the coach's changes do what they expected? Would a run with the coach switched off look different?

Research tried 28 strategies; the coach applied 16 changes. Costs 5 bp per unit of weight traded. Partial seasons use data up to the last available day.

Run it yourself

Everything is public. Clone it, pull one model, open the page

Three commands

```
ollama pull qwen3:8b
git clone https://github.com/jimkliew/aiInvesting.git && cd aiInvesting
uv run python -m aitrader serve          # http://localhost:8000, click Run campaign

# or the command line
uv run python -m aitrader campaign --quick               # 2 short seasons, ~2 min
uv run python -m aitrader campaign --personas momentum,contrarian --apply approve
uv run python -m aitrader replay runs/<campaign> --speed 5
uv run pytest                                            # 52 offline tests
```

WHICH MODEL FOR WHICH LAPTOP

8 GB	`qwen3:4b`	slower; run 3 seasons; use Replay in class
16 GB	`qwen3:8b`	the default, verified
32 GB+	`qwen3:14b`	better code, about twice as slow

IN THE REPO

`docs/superpowers/specs/`

The full design: roles, phases, schemas, lever menu, budgets, error handling, test plan, two-day build schedule.

`docs/mockups/mission-control.html`

The dashboard you just saw. One file, open it in any browser, no server needed.

`aitrader/`

The package: orchestrator, agents, prompts, levers, sandbox, engine, web server. 52 tests run offline with a fake model.

No secrets, by design

Nothing in the repo needs a key, so there is nothing to leak. The .gitignore excludes data and run folders. MIT license.

https://github.com/jimkliew/aiInvesting open source under the MIT license

Published Oct 24, 2026: design spec, dashboard and the Python package are in the repo. 52 offline tests; verified end to end against Ollama with real data.

Questions for the room

Argue from the numbers the dashboard shows

Six seasons of data, seven agents, one coach. What would convince you the team is skilled rather than lucky?

1 Did validation predict the season?

Look at the hollow and filled dots. If a strategy's validation Sharpe does not predict its season Sharpe, what is the selector actually selecting?

2 Would you let the coach apply changes automatically?

Auto mode closes the loop. Approve mode makes you the risk committee. Which would a real fund choose, and why?

3 Where could lookahead still sneak in?

The engine lags one bar, the sandbox blocks negative shifts, the perturbation test withholds 60 days. What did we miss?

4 Is a 50/50 team blend fair?

The headline number is half quant book, half discretionary desk. Should the headline be whichever did better, or neither?

Open the dashboard at any point. Every number on these slides is one click away in Mission Control.